Asymmetric Bonding and Buyer Dominance: A Verified Solidity Implementation

Alessandro Daliana

April 2026 (snapshot: 2026-04-23)

Abstract

We describe a reference Solidity implementation of the two-mechanism bonded commitment settlement primitive — *asymmetric bonding* (each party locks 2× collateral, with the seller bonding against cumulative upstream value) and *buyer dominance with atomic resolution* (only the root buyer may extend or resolve, and resolution settles every active order in the process simultaneously or not at all) — together with the formal-verification stack used to audit it. The kernel (`FigaroCore`) is **ownerless, fee-less, and admin-less**: two external functions, three storage mappings, no upgrade path, no escape hatch from the bonded state. Verification is layered: TLA$^+$ model checking exhaustively explores 6,087,113 distinct states across seven kernel invariants; Echidna fuzzes seven property invariants under a 50,000-call campaign budget; Halmos symbolically proves seven kernel properties via the z3 SMT solver; Certora cloud proves twenty-two declared CVL rules (twenty-three sub-rules in the cloud report) across the kernel, the attestation surface, and a token-operations conservation surface that gates every ERC-20 call site in the source tree.

The implementation also realizes a *coordinator pattern*: an extension discipline under which external mechanisms (Dutch auction, schema-typed attestation coordinator, schema and operator registries) compose with the kernel without weakening the bonding equilibrium. We give the four sufficient conditions and three concrete instances. Finally, we describe the three-layer enforcement architecture—economic (bonding), coordinational (atomic resolution), evidentiary (immutable log)—and the threat model under which the layers compose.

This paper is a snapshot at commit-of-record 2026-04-23. Verification counts will grow as the test surface expands; the methodology is the stable artifact.

Keywords: smart contracts, formal verification, TLA$^+$, Halmos, Certora, Echidna, EIP-712, settlement layer, coordinator pattern

1 Introduction

The bonded commitment settlement primitive composes two mechanisms. *Asymmetric bonding*: for an order with payment P and cumulative upstream value $G \geq P$, the buyer locks $2P$ and the seller locks $2G$, so cooperation is the weakly dominant strategy at every position in an N-party process chain and the cooperative profile is the unique outcome surviving iterated elimination of weakly dominated strategies. *Buyer dominance with atomic resolution*: only the root buyer may extend or resolve a process, and resolution settles every active order in the process simultaneously or not at all, inducing a weakest-link subgame among co-sellers under which cooperation pressure propagates without explicit communication.

The mechanism is settlement-substrate-agnostic; it admits realisation on any state machine that maintains a monotonic cumulative-value accumulator and authenticates buyer signatures. This paper presents *one* such realisation — a Solidity smart contract `FigaroCore` on the EVM — and the verification methodology applied to it.

The core property the verification is asked to deliver is precisely the gap between mechanism and code: the equilibrium analysis assumes the settlement layer enforces (i) the asymmetric

bond formula $C_b = 2P$, $C_s = 2G$ on `commit` with a monotonic accumulator, and (ii) buyer dominance with atomic resolution on `resolveProcess`. The code must *actually* enforce those, in every reachable path, against all reasonable adversaries. Bridging that gap is the work of the verification stack.

What verification does and does not deliver. A distinction worth naming up front: the equilibrium argument is a property of rational play over a payoff structure — cooperation weakly dominates defection at every position in an N-party process chain, under specified rationality assumptions — and it is not a property of code. The four verification tools we apply do not verify the equilibrium itself; they verify the *structural preconditions* that the equilibrium assumes (asymmetric bond formula, monotonic accumulation, buyer dominance, atomic resolution, conservation of value). What we verify is that the code implements the payoff structure faithfully in every reachable path. Claims that some external mechanism "preserves the kernel's bonding equilibrium" invoke the equilibrium argument (which holds by rational play) and not the verification claim made here (which holds by code inspection). Keeping the two claims distinct matters for any reader who wants to know exactly which property is backed by which evidence.

Brief recap of the settlement primitive. The two mechanisms together yield: for each committed order, the buyer locks $2P$ and the seller locks $2G$; only the root buyer can trigger resolution; on resolution the buyer recovers P and each seller recovers $2G_i + P_i$, with every order in the process settling simultaneously or none. This is the full state-machine surface that the kernel must enforce; the game-theoretic derivation (two-party Nash, N-party generalisation, escape-hatch impossibility, the reduction over joint-liability microfinance, and the marginality of dispute resolution) is mechanism-design content and lies outside the scope of the present implementation paper.

Paper organization. Section 2 describes the contract architecture and the EIP-712 commitment protocol. Section 3 catalogues the features deliberately absent from the kernel and ties each absence to a mechanism property it would have broken. Section 4 describes the four-layer verification methodology. Section 5 presents the current results. Section 6 develops the threat model and three-layer enforcement architecture. Section 7 formulates the coordinator pattern and exhibits three concrete instances. Section 8 concludes.

2 Smart Contract Architecture

The kernel is a single Solidity 0.8.26 contract, `src/FigaroCore.sol`, inheriting OpenZeppelin's `EIP712` and `ReentrancyGuard`. It exposes two external functions: `commit` and `resolveProcess`. There is no constructor parameter, no owner, no admin, no upgrade path.

2.1 Storage Layout

Three mappings comprise the kernel state:

- `processes`: `bytes32` → `ProcessState`, where `ProcessState` is a struct of `(address rootBuyer, IERC20 currency, uint256 cumulativeValue, uint256 activeOrderCount)`.
- `orderStatus`: `bytes32` → `uint8` encoding $\{0 : \text{unknown}, 1 : \text{committed}, 2 : \text{resolved}\}$. The status function is monotonically non-decreasing; this is one of the verified invariants.
- `orderProcessId`: `bytes32` → `bytes32` recording each order's process membership.

The kernel deliberately does not store a per-order struct. Order terms (payment, cumulative-value snapshot, parties, denomination) are reconstructed off-chain from the signed commitment and the emitted `OrderCommitted` event. Bond amounts ($2P$ for the buyer, $2G$ for the seller) are deterministic functions of those values rather than separate storage variables. Storage is therefore $O(\text{processes}) + O(\text{orders})$ uint8 + $O(\text{orders})$ bytes32, with no order-payload duplication.

2.2 Commitment Protocol

Both parties sign an EIP-712 typed commitment [EIP-712, 2017] off-chain. A single on-chain `commit` call verifies both signatures (via `ECDSA.recover` against the EIP-712 digest) and pulls both bonds atomically. Root commitments create a new process; sub-order commitments extend an existing one by carrying the inherited `processId` and the next expected cumulative value. This eliminates the accept-reject pattern of traditional escrow protocols (which front-run on acceptance) and makes the act of committing simultaneous from the chain's point of view.

2.3 Identifier Derivation

Process and order identifiers are content-addressed:

$$\text{processId} = H_{\text{EIP-712}}(\text{root commitment}), \tag{1}$$

$$\text{orderHash} = \text{keccak256}(\text{processId} \parallel \text{structHash}). \tag{2}$$

The EIP-712 domain separator binds the root identifier to chain id and verifying contract, preventing replay across deployments. Sub-orders inherit `processId` from the signed commitment. No auto-incrementing counter exists; identifier collisions require SHA-3 collisions.

2.4 Fee-on-Transfer Protection

The internal helper `_pullExact(IERC20 token, address from, uint256 amount)` reads the contract's pre-transfer balance, calls `safeTransferFrom`, and reverts with `FeeOnTransferDetected` if `balanceAfter` − `balanceBefore` ≠ `amount`. This ensures the conservation law cannot be broken by tokens that apply a transfer tax—a class of issues that has silently corrupted accounting in several DeFi systems.

2.5 Events

Five events are emitted, supporting full off-chain state reconstruction without a subgraph dependency:

- `OrderCommitted` carries the canonical commitment payload — ten fields: `orderHash`, `processId`, `buyer`, `seller`, `currency`, `payment`, `cumulativeValue`, `agreementHash`, `salt`, `deadline`. The `salt` and `deadline` fields are what enable the SDK's `reconstruct()` primitive to derive the EIP-712 digest deterministically from event logs alone.
- `OrderSeller` and `OrderCurrency` are companion events that index the seller and currency respectively—needed because the EVM caps indexed event arguments at three.
- `OrderResolved` fires once per order at resolution.
- `ProcessResolved` fires once per process at resolution, with the order count.

A reader who replays the four event types into a state machine exactly reconstructs `processes`, `orderStatus`, and `orderProcessId`—this is the basis of the SDK's `reconstruct()` primitive used by agents and frontends.

3 Design Non-Decisions

The following features are deliberately absent. Each absence preserves a mechanism-design property of the bonded commitment primitive (asymmetric bonding, buyer dominance, atomic resolution, or the no-escape-hatch security constraint).

1. **No owner, admin, or upgrade.** There is no `pause()`, `upgrade()`, or `transferOwnership()`. The contract is ownerless from deployment. Adding any of these would introduce an unbonded actor (the owner) into the resolution path, which the escape-hatch impossibility theorem rules out.

2. **No protocol fee.** Settlement distributes the full bond amount to the parties. A fee at $k = 2$ would shift the buyer's net cooperation payoff above the defection payoff only by $P -$ fee, narrowing the dominance margin and — if the fee approaches P — collapsing the equilibrium.

3. **No timeout.** An active order remains active indefinitely. Per the escape-hatch impossibility theorem, any timeout with recovery fraction $\alpha \geq \frac{1}{2}$ destroys weak dominance for the buyer.

4. **No partial resolution.** The resolution function requires the full active order list and reverts otherwise. This enforces atomicity, on which the weakest-link coordination pressure depends.

5. **No internal ledger.** Payouts are direct `safeTransfer` calls, not balance increments to be withdrawn later. This eliminates withdrawal-pattern reentrancy surface and removes a class of accounting drift bugs.

6. **No restriction on buyer–seller equality.** The contract does not forbid $B = S$. If a single address signs both sides of a commitment, that address deposits both bonds and receives both payouts at resolution; the bond math is self-cancelling and no third party is exposed. We surface this explicitly because it is a reasonable design question to ask of every smart-contract escrow: in `FigaroCore`, the answer is that the equilibrium analysis treats $B \neq S$ as the standard case but the contract permits the degenerate case rather than introducing a guard whose effect would be cosmetic.

3.1 Custom Errors as Specification

The kernel defines fifteen custom errors, each named for the invariant it protects: `DeadlineExpired`, `InvalidBuyerSignature`, `InvalidSellerSignature`, `ZeroPayment`, `ProcessAlreadyExists`, `UnknownProcess`, `CumulativeValueMismatch(uint256 expected, uint256 actual)`, `NotProcessBuyer`, `CurrencyMismatch`, `OrderNotCommitted(bytes32 orderHash)`, `NoActiveOrders`, `IncompleteOrderList(uint256 required, uint256 provided)`, `DuplicateCommitment`, `FeeOnTransferDetected`, `InvalidRootCumulativeVa`
The error names function as a partial executable specification: each error is exercised by a Foundry revert-branch test and referenced by name in the verification-results section below.

4 Formal Verification Methodology

We use four verification techniques, each chosen for what it covers that the others do not.

TLA$^+$ model checking (TLC). Captures the full state machine as a single transition system. Invariants are propositional safety properties. TLC explores all reachable states under bounded parameters by breadth-first search, exhibiting either an invariant violation with a minimal trace

or exhaustive coverage of the bounded space. Strength: high-level state-machine reasoning; abstracts cryptography and ERC-20 mechanics to expose accounting errors. Weakness: bounded by parameters; gives no guarantee for unbounded state.

Property-based fuzzing (Echidna). Runs randomized call sequences against the deployed bytecode under HEVM, with EIP-712 signing performed via cheatcodes. Strength: exercises real bytecode against concrete adversaries, including combinations of operations the model abstracts. Weakness: random sampling; finds bugs but does not prove their absence.

Symbolic execution (Halmos, z3-backed). Encodes call traces as SMT formulas and asks z3 whether any concrete input satisfies the negation of an invariant. Strength: when it returns *verified*, the property holds for *all* inputs in the modeled trace; complements Echidna's bounded random sampling with symbolic coverage of the commit/resolve state machine. Weakness: symbolic complexity blows up with control-flow depth; tractable on bounded path lengths.

Specification checking (Certora). SMT-based proving of CVL rules against the bytecode, run on Certora's cloud service. Strength: rules can quantify over methods (`method f`), allowing universal statements like "no method modifies X". Weakness: paid service; rule encoding is non-trivial.

Foundry as the regression spine. A Foundry test suite (31 files, 420 tests as of snapshot date) serves as the continuous-integration spine. Foundry tests are not counted toward formal verification but provide the harness the other three techniques compile against.

5 Verification Results

All numbers in this section are reported against the snapshot at 2026-04-23, repository commit `970ee3a914b2`[1]. Re-runs produce different fuzz-call counts; the configuration parameters (invariants, rule counts, model bounds) are the stable artifact.

5.1 TLA$^+$ (`formal/FigaroCore.tla`)

The contract is modeled in TLA$^+$ with eight state variables (`processes`, `orderStatus`, `orderRecords`, `processOrders`, `contractBalance`, `wallets`, `nextProcId`, `nextOrdId`) and three actions: `CommitRoot`, `CommitSub`, `ResolveProcess`. The model abstracts EIP-712 signatures (assumed correct given valid commitments), ERC-20 mechanics (modeled as integer balances), and timing (deadlines are orthogonal to the bonding equilibrium and are exercised in Foundry instead).

Model bounds (`formal/MC.cfg`). 2 buyers (b_1, b_2), 2 sellers (s_1, s_2), initial balance 30, maximum payment 3, 2 concurrent processes, 2 sub-orders per process. Two buyers (rather than one) enable cross-buyer invariant verification—a single-buyer model would not exercise `NotProcessBuyer` branches reachable by an attacker buyer.

Invariants verified. Seven safety invariants verified exhaustively:

I1 TypeOK: type well-formedness of all state variables.

I2 TokenConservation: sum of wallet balances plus contract balance equals total supply.

[1]Pin to this commit when reproducing to guarantee identical bytecode; later commits may change call counts as the test surface evolves.

I3 ContractSolvency: `contractBalance` ≥ 0.

I4 WalletNonNegative: all participant balances ≥ 0.

I5 CumulativeIntegrity: per-process, G equals the sum of all order payments.

I6 ActiveCountCorrect: stored active-order count matches the count of committed orders.

I7 ResolutionAlwaysPossible: for every active process, the contract holds sufficient funds to resolve all orders.

Result. 6,087,113 distinct states explored in 4 minutes 8 seconds; zero invariant violations. Reproduced via `./test-tla.sh`.

5.2 Echidna (`src/echidna/EchidnaFuzzer.sol`)

Seven property invariants, each prefixed `echidna_`, checked after every call sequence:

E1 `solvency`: token balance $\geq$ sum of outstanding bonds.

E2 `active_count_consistent`: stored count equals actual.

E3 `cumulative_accounting`: accumulator equals sum of payments.

E4 `state_monotonicity`: orders never transition backwards ($0 \to 1 \to 2$, never reverse).

E5 `token_conservation`: total supply constant under all commit/resolve sequences.

E6 `buyer_dominance`: non-buyer `resolveProcess` always reverts.

E7 `atomic_resolution`: incomplete order list always reverts.

Configuration (`echidna.yaml`). `testLimit: 50000` (campaign budget), `seqLen: 30`, `shrinkLimit: 5000`, three deployer/sender accounts, property mode, 300-second wall-clock timeout. The Echidna campaign budget is the reproducibility anchor; raw call counts vary between runs.

Result. All seven properties held across the campaign; the most recent run exercised approximately 43,000 individual calls before the budget was exhausted, with no counterexample produced. Reproduced via `./test-echidna.sh`.

5.3 Halmos (`test/HalmosFigaroCore.t.sol`)

Seven kernel properties, each implemented as a `check_` function and discharged by z3 under symbolic-input quantification:

H1 `tokenConservation_afterCommit`.

H2 `contractSolvency_afterCommit`.

H3 `correctBondAmounts`: $C_b = 2P$ and $C_s = 2G$ for all symbolic P.

H4 `resolutionPayouts`: $\pi_s = 2G + P$ and $\pi_b = P$ for all symbolic P.

H5 `orderStatusTransition`: status moves only $0 \to 1 \to 2$.

H6 `buyerDominance_revert`: any non-buyer caller of `resolveProcess` reverts.

H7 `cumulativeValueMonotonic`: across two symbolic sub-order commitments, G never decreases.

Result. All seven discharged by z3. Reproduced via the Foundry profile `halmos` as documented in the repo.

5.4 Certora (`certora/`)

Five specification files. We list the rule counts per surface and the headline rule for each.

Kernel (`FigaroCore.spec`, 8 rules). `orderStatusNeverDecreases`, `orderStatusTransitionsAreValid`, `commitIncreasesActiveCount`, `onlyBuyerCanResolve`, `noDoubleCommit`, `cumulativeValueMonotonic`, `rootBuyerImmutable`, `currencyImmutable`.

Attestation surface (`AttestationCoordinator.spec`, 7 rules). Two role-gate rules: `nonBuyerCannotAttes` `successfulBuyerAttestationImpliesBuyer`. Two parametric Core-immutability rules (the attestation surface cannot change kernel state, quantified over every public function on the coordinator): `attestationCannotChangeOrderStatus`, `attestationCannotChangeProcessState`. Three validator-gate rules (verified on Certora cloud 2026-04-23): `noValidatorBlocksBuyerAttestation`, `setValidatorIsFirstWriteWins`, `setValidatorPreservesOtherBindings`.

Token-operations conservation (`TokenOpsVerification.spec`, 7 declared rules). `commitBuyerExactDe` `commitSellerExactDelta`, `commitContractExactDelta`, `commitAllowanceDrainSafety`, `commitTokenConserv` `resolveSingleOrderPayout`, `resolveSingleOrderConservation`. A regression guard for "no untracked token moves" is documented in the spec but is not itself a CVL rule. Certora's prover splits `resolveSingleOrderConservation` into two sub-cases, so the service report shows 8 sub-rules green for this spec. The token-ops surface is gated by a pre-run lint (`lint-token-ops.sh`, invoked from `test-certora.sh`) that asserts the spec's inventory file enumerates every ERC-20 `transfer`/`transferFrom`/`safeTransfer*` call site in `src/`. New transfer call sites without a matching inventory entry fail before any cloud dispatch.

Other specs. `StagedMerkleAirdrop.spec` (3 rules) covers the airdrop contract, and `FigToken.spec` (6 rules) covers the FIG token registry. Both are out of scope for the kernel claim of this paper but ship in the same repository and are verified together.

Total Certora kernel-relevant rules. $8 + 7 + 7 = 22$ declared rules across three specs. Certora's prover splits `resolveSingleOrderConservation` into two sub-cases, so the cloud report shows 23 sub-rules green for the kernel-relevant total. All discharged on Certora cloud as of the snapshot date. The repository ships a sixth spec (`BatchVerifierTokenOps.spec`, 4 rules) covering the optional batch-settlement path; it is verified together with the kernel specs but is out of scope for the kernel claim of this paper, so the kernel-relevant total is 22 across three specs while the full repository total is 35 across six specs.

5.5 Foundry Regression Spine

31 test suites, 420 tests (including `FigaroCoreRevertBranchTest` which targets every custom error listed in Section 3; `ParityVectors` which asserts EIP-712-digest parity between the SDK's TypeScript hash function and the on-chain implementation; and `FigaroCoreEventEmissionTest` which asserts every event field encodes as the SDK expects).

5.6 Scope of the Verification Claim

What this paper's verification establishes, in plain language for non-specialist readers: the Solidity source at commit `970ee3a914b2` has been audited by four different methodological ap-

proaches, and each approach has found that the source satisfies seven named properties within the bounds each approach explores. What this does *not* establish:

- **That the specification is the right specification.** Verification checks code against a spec; it does not check whether the spec captures what the author meant or what users expect.
- **That bytecode at a deployed address was compiled from this commit.** The kernel's production deployment is a separate act, performed by a separate party, using a compiler under settings and dependency versions that this paper does not pin. Reproducing the verified bytecode against a deployed address requires standard contract-source-verification tooling (Etherscan-style verification, sourcify, or reproducible-build pipelines) which is out-of-scope here. Relying parties should confirm the source-to-bytecode-to-address chain independently.
- **That the denomination token behaves.** The kernel takes an arbitrary `IERC20`. If the chosen token's issuer exercises blocklist, freeze, upgrade, or pause authority over a party's bonded balance, the "no escape hatch" property of the kernel is vacuously violated — the bond is still locked from the kernel's perspective, but it is unreachable from the world's. Parties should treat the choice of denomination token as a selection of whose escape hatches to accept, not as an avoidance of escape hatches in aggregate.
- **That chain liveness and key custody persist.** If the host chain halts, censors, or reorgs past finality, and if either party loses or has compromised their signing key during the process, the kernel has no recovery mechanism. These are operational-security dependencies that sit above the verified surface.

The honest summary is that verification reduces one class of risk (specification–implementation gap within the verified bounds) and displaces several others (specification correctness, deployment chain of custody, token-issuer authority, host-chain liveness, key custody) onto layers the paper does not itself verify. A practitioner or legal reader consuming these verification claims should treat them as input to due diligence, not as a substitute for it.

6 Security Analysis

6.1 Threat Model

We assume rational adversaries who maximize economic payoff. We consider four attack vectors.

Griefing (buyer refuses to resolve). The attacker's cost is $2P$ (locked bond) plus opportunity cost on that capital plus permanent on-chain reputation damage. The attacker's benefit is zero. For rational agents, the attack is strictly dominated by cooperation. For irrational agents (spite exceeds economic loss), Layer 3 (below) provides deterrence: the immutable on-chain record serves as evidence in off-chain legal proceedings.

Sybil (fake orders to manipulate state). Each order requires real bond deposits totaling $2(G_i + P_i)$ per order. At scale, Sybil attacks are capital-intensive with no path to extracting the invested capital—there is no secondary market for locked bonds, no yield, no governance influence.

Front-running. Order identifiers are content-addressed from the dual-signed commitment. An attacker who observes a pending `commit` transaction cannot create a conflicting order for the same process: producing a different signature changes the digest, which changes the order hash, so the front-runner's order would be a distinct order, not a conflicting one. No extractable MEV exists.

Cumulative value manipulation. Overstating cumulative value is strictly dominated under the bond formula $C_s = 2G'$: a seller who reports $G' > G_{\text{true}}$ posts a larger bond ($2G'$) for the same payment recovery P, leaving expected utility strictly decreasing in the reported value (the mechanism's *cumulative-value reporting honesty* result). Understating is prevented by the contract's monotonic accumulator check (`CumulativeValueMismatch` revert).

6.2 Liveness

Theorem 6.1 (Liveness Under Rationality)**.** *Let B be a buyer with time-discounted utility $U_B = \sum_{t=0}^{\infty} \delta^t \cdot u_t$ where $\delta \in (0,1)$ is the discount factor and u_t is the per-period payoff. If all sellers have cooperated (the buyer has received the agreed goods), then the buyer resolves in finite time.*

Proof. The buyer's payoff under *resolve at* t^* is $\delta^{t^*} \sum_i P_i$; under *never resolve* is 0. Since $\delta \in (0,1)$ and $P_i > 0$, every finite t^* strictly dominates non-resolution; the optimum is $t^* = 0$. □

Remark 6.2 (Limitations). Theorem 6.1 assumes (i) the buyer received satisfactory performance, (ii) $\delta < 1$, and (iii) no spite. Condition (iii) is the hardest to guarantee. When it fails, the mechanism cannot guarantee liveness by economic incentives alone—the residual case is addressed by Layer 3 (legal deterrence via immutable on-chain evidence).

6.3 The Three-Layer Enforcement Architecture

Table 1: Defense-in-depth: three enforcement layers.

Layer	Mechanism	Coverage	Failure Mode
1. Bonding	Asymmetric collateral	Rational actors	Irrational actors
2. Coordination	Atomic resolution	Multi-seller failures	Uncoordinated sellers
3. Evidence	Immutable on-chain log	Adversarial actors	Jurisdictional limits

Layer 1 (economic). The bonding equilibrium handles rational participants: cooperation strictly dominates defection at every node under the asymmetric bond formula.

Layer 2 (coordinational). Atomic resolution induces the weakest-link subgame; honest sellers have direct economic interest of magnitude $P_i + 2G_i$ in ensuring co-sellers cooperate. No explicit communication is required.

Layer 3 (evidentiary). Every commitment, every bond deposit, every resolution (or its absence) is recorded with block timestamps. The kernel does not attempt on-chain dispute resolution; it provides the *evidence* that off-chain dispute systems already accept.

6.4 Scope and Limitations

Liveness under non-rational failure. Catastrophic non-rational failure (lost keys, hardware failure, participant death) imposes losses on co-sellers via atomic resolution regardless of incentive alignment. The coordination-pressure result assumes payoffs depend on co-sellers' *choices*; failure is not a choice. Operational reliability under such failure depends on participant continuity practices (key management, multisig, delegated signing) that sit above the kernel and are out of scope.

Capital opportunity cost. At prevailing risk-free rate r over process duration t, the buyer and seller forgo rPt and $2rGt$ respectively. Dominance is preserved at any $rt > 0$, but the operational margin narrows; the $2\times$ multiplier is sufficient as a matter of game-theoretic derivation, but the practical applicability favors processes short relative to the prevailing risk-free horizon.

7 Mechanism Composition: The Coordinator Pattern

The kernel's bonding equilibrium is a fixed point. The natural next question—under what conditions does an external mechanism composed with the kernel preserve the equilibrium?—is answered, in this implementation, by the *coordinator pattern.*

Proposition 7.1 (Coordinator Pattern — Sufficient Conditions)**.** *An external mechanism M composed with* `FigaroCore` *preserves the kernel's bonding equilibrium if it satisfies:*

(i) ***Read-only kernel access****: M interacts with kernel state via* `staticcall` *or event consumption only.*

(ii) ***Role non-impersonation****: M never calls* `commit` *or* `resolveProcess` *on behalf of a buyer or seller; signatures are originated by the parties themselves.*

(iii) ***Bond non-modification****: M holds no kernel bonds and does not interpose between the parties and the bond transfers.*

(iv) ***No bypass****: M does not provide an alternative resolution path or escape from* `Active`*.*

Theorem 7.1 is sufficient, not necessary; mechanisms that violate one of (i)–(iv) may still preserve the equilibrium under additional argument, but those arguments are per-mechanism. The four conditions are currently checked by inspection per extension contract rather than by a parametric Certora rule that quantifies over arbitrary M — see Section 8 for the verification status of this proposition. The envelope is the discipline that makes composition *by construction* safe. We exhibit three instances.

Dutch auction (`src/DutchAuction.sol`). A descending-price coordination primitive that performs role allocation: who may become the seller for a given root commitment. The auction holds no tokens, custodies no bonds, and never calls `commit`. The auction's winner subsequently signs an EIP-712 commitment off-chain and the parties execute `commit` themselves. Conditions (i)–(iv) hold by construction.

Attestation coordinator (`src/AttestationCoordinator.sol`). A unified zero-storage attestation surface, validator-gated per `schemaId`. Three modes: `attestAsSeller`, `attestAsBuyer`, `attestViaResolver`. Each call verifies role membership against the kernel's `ProcessState.rootBuyer` (or against an `IRoleResolver` for the resolver path), runs `schemaValidator[schemaId].validate(...)` on the content, and emits an `Attestation` event whose `contentRef` is keccak256(content). The coordinator never modifies kernel state—this is the headline Certora rule `attestationCannotChangeOrderStatu` and its sibling `attestationCannotChangeProcessState`, both verified exhaustively. Conditions (i)–(iv) hold; the coordinator is the cleanest example of an event-only extension.

Schema and operator registries (`src/SchemaRegistry.sol`, `src/OperatorRegistry.sol`). Permissionless, append-only event-first registries. The schema registry anchors content schemas as `keccak256(name)` with a URI hash pointing at the off-chain JSON spec. The operator registry is on-chain operator self-registration with a reclaimable ETH deposit. Both compose with the kernel by being visible to readers and not being callable by `commit` or `resolveProcess`. Conditions (i)–(iv) hold; both registries are pure coordination surfaces with no kernel interaction at all.

What this gives. Theorem 7.1 is the discipline used to evaluate every proposed extension. A mechanism that satisfies all four conditions can be composed without re-doing the equilibrium proof; a mechanism that violates one of them requires explicit mechanism-level argument and re-verification. In the reference implementation, all five extension contracts shipped to date satisfy Theorem 7.1 by construction.

A note on liability allocation when conditions fail. The verification status of Theorem 7.1 is important for legal readers: the four conditions are presently checked by inspection per extension contract rather than by a parametric Certora rule that quantifies over arbitrary M (see Section 8 for the verification gap this leaves open). If an extension contract silently violates condition (i), (ii), (iii), or (iv) and a downstream user is harmed when the kernel equilibrium breaks for that composition, the kernel verification in this paper does not extend to the resulting harm. Liability in such a case is allocated by the legal regime governing the extension deployer's conduct, the audit artifacts (or their absence) accompanying the extension, and the disclosures made to users — all of which sit outside the kernel and outside the present paper's scope. The kernel claims it is correct; it does not claim that every extension built against it is correct, nor does it claim that the legal chain from kernel correctness to user remedy is closed.

8 Conclusion

We have presented a verified Solidity implementation of the two-mechanism bonded commitment settlement primitive: an ownerless, fee-less kernel of two functions and three mappings, with a four-layer verification stack (TLA$^+$, Echidna, Halmos, Certora) that exercises the same invariants from four methodological directions. The coordinator pattern of Section 7 provides a discipline for extending the kernel without weakening its equilibrium.

The deeper claim: defense in depth, not a probability bound. Beyond the headline numbers, the value of the verification stack is that four methodologically different proofs converge on the same seven properties (token conservation, solvency, bond correctness, status monotonicity, buyer dominance, cumulative monotonicity, atomic resolution). We present this as *defense in depth*: a bug that survives all four toolchains must be consistent with the bounded-exhaustive exploration of TLA$^+$, the property-level random sampling of Echidna, the symbolic-input discharge of Halmos, and the method-quantified SMT rules of Certora. We do not attempt a formal probability bound on residual error: Halmos and Certora both lean on the z3 SMT solver, which breaks the independence assumption any probabilistic bound would require, and the prior distribution over kernel-invariant violations in this toolchain ecosystem is not characterized in the literature. The honest statement is that four methodologically distinct proofs agreeing provides substantially stronger assurance than any one alone, not that the residual probability is bounded above by any particular value. The stack reduces risk; it does not retire it.

Acknowledgments

The verification surface described here owes its current shape to many hands, and in particular to the maintainers of TLA$^+$, Echidna, Halmos, and Certora, whose tooling made the multi-method verification stack practical to assemble.

References

Bloemen, R., Logvinov, L., and Evans, J. EIP-712: Ethereum typed structured data hashing and signing. Ethereum Improvement Proposal, 2017. `https://eips.ethereum.org/EIPS/eip-712`